西安電子科技大學

《计算机操作系统课程设计》

课 程 报 告

题　　目　实验四：Priority-Scheduling—Task2

指导教师　李海昆

学　　院　计算机科学与技术学院　专　　业　计算机科学与技术

学　　号　19050500032　班　　级　1903011

姓　　名　许珈铭

完成时间　2021 年 12 月 23 日

目录

1 实验目的 **1**

2 实验原理 **1**

2.1 pintos 系统内核代码理解 1

2.2 基本原理 2

3 实验方案 **5**

3.1 环境配置 5

3.2 正式实验 9

3.3 实验结果 14

4 实验心得 **15**

实验四：Priority-Scheduling—Task2

1　实验目的

1. 在优先级调度的基础上，加入优先级捐赠机制，防止系统在高优先级线程所需资源被低优先级线程占有时出现优先级反转问题。
2. 理解 threads.c、threads.h 文件中有关线程的优先级的结构体、函数的含义与作用
3. 解决由 *lock* 造成的优先级翻转的问题。
4. 本实验要求为 Pintos 建立优先级捐赠机制，熟悉优先级捐赠算法。

2　实验原理

2.1　pintos 系统内核代码理解

1. *../pintos/src/threads/* 目录下的 *thread.h* 和 *thread.c* 文件是有关线程初始化、阻塞、解除阻塞，线程调度等内容。
   可以通过输入 `geany thread.c` `geany thread.h` 命令用 *geany* 文本编辑器来打开相应的文件查看内容。当然也可以使用 *vim* 或 *gedit* 文本编辑器来替换 *geany*。

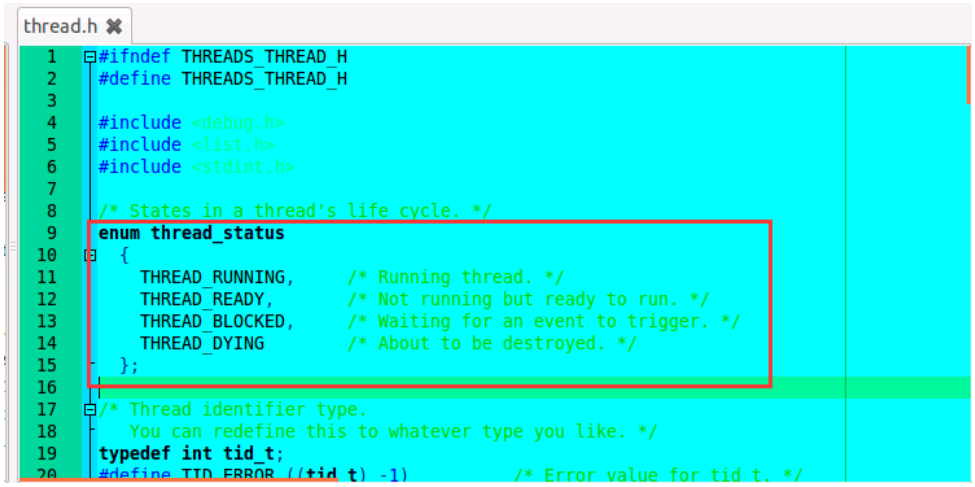


图 1: thread.h 文件

- 理解 *thread.h* 文件中的 *PRI_MIN*、*PRI_DEFAULT*、*PRI_MAX* 宏定义。

```c
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                      /* Name (for debugging purposes). */
    uint8_t *stack;                     /* Saved stack pointer. */
    int priority;                       /* Priority. */
    struct list_elem allelem;           /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;              /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic;                     /* Detects stack overflow. */
  };
```

图 2: thread 结构体内容

tid	该线程的编号
thread_status	该线程目前的状态（enum 类型）
name[16]	该线程的名字
priority	该线程的优先级
...	...

其中 *priority* 代表的是该进程的优先级，在 *thread.h* 文件中已经定义了线程优先级的范围，如下图所示：

```c
/* Thread priorities. */
#define PRI_MIN 0                       /* Lowest priority. */
#define PRI_DEFAULT 31                  /* Default priority. */
#define PRI_MAX 63                      /* Highest priority. */
```

图 3: 线程优先级范围

2.2 基本原理

1. 优先级反转问题

   - 定义 (非标准定义): 操作系统中优先级翻转问题指当一个高优先级任务通过信号量机制访问共享资源时，该信号量已被一低优先级任务占有，而这个低优先级任务在访问共享资源时可能又被其它一些中等优先级任务抢先，因此造成高优先级任务被许多具有较低优先级任务阻塞并滞后执行的现象。
   - 优先级反转发生的条件:
     - 至少需要有三个任务，它们的优先级分别是高、中、低。
     - 低优先级和高优先级任务争夺同一个锁/信号量。

- 危害: 优先级反转会造成任务调度的不确定性，在实时系统中，还可能导致严重错误甚至系统崩溃。

2. 解决方案

- 优先级天花板 (Priority ceiling)(本实验不需要实现)：
  - 当任务申请某资源时，把该任务的优先级提升到可访问这个资源的所有任务中的最高优先级，这个优先级称为该资源的优先级天花板。
  - 这种方法简单易行，不必进行复杂的判断，只要任务访问共享资源都会提升任务的优先级。但这种不分青红皂白直接提升优先级的方法会使受影响的线程数量大大增加，降低了系统的效率。
- 优先级继承 (Priority inheritance)(本次实验需要实现)：
  优先级继承 (又称优先级捐献) 主要分为以下几种：
  - 简单捐献

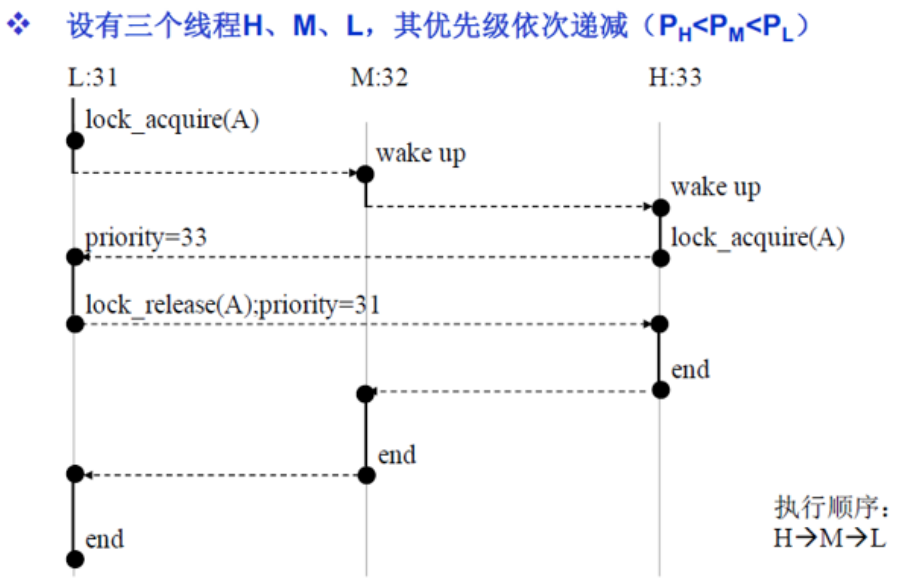


图 4: 简单捐献

  - 多重捐献

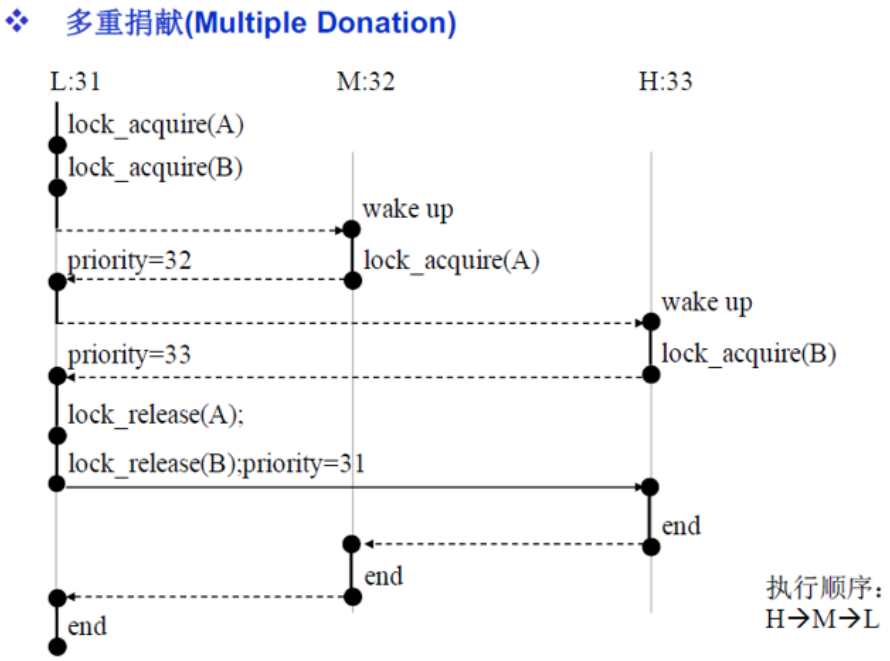


图 5: 多重捐献

– 递归捐献

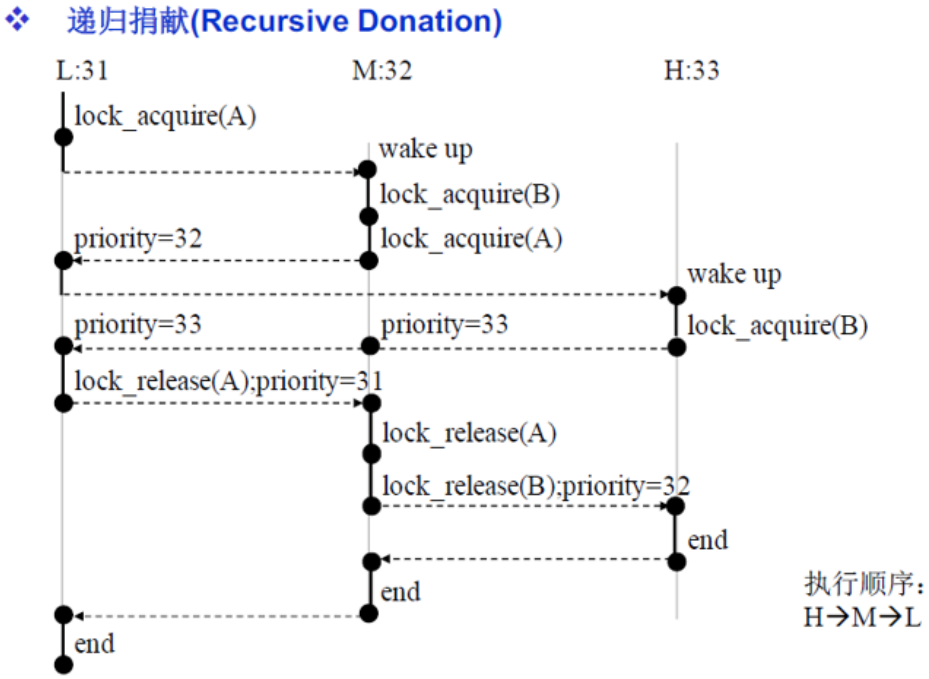


图 6: 递归捐献

3 实验方案

3.1 环境配置

3.1.1 云服务器的连接

- 实验工具：阿里云服务器、Xshell 远程连接软件、VNC Viewer。
- 实验步骤：按如下步骤执行即可

1. 打开 Xshell 软件，点击新建

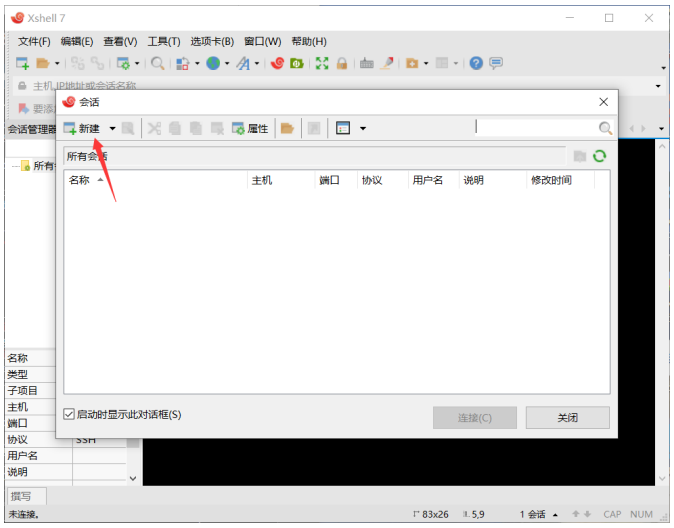


图 7: Xshell 软件界面-1

2. 在阿里云网站上申请相应的服务器资源后，按照图中指示顺序填写相关内容

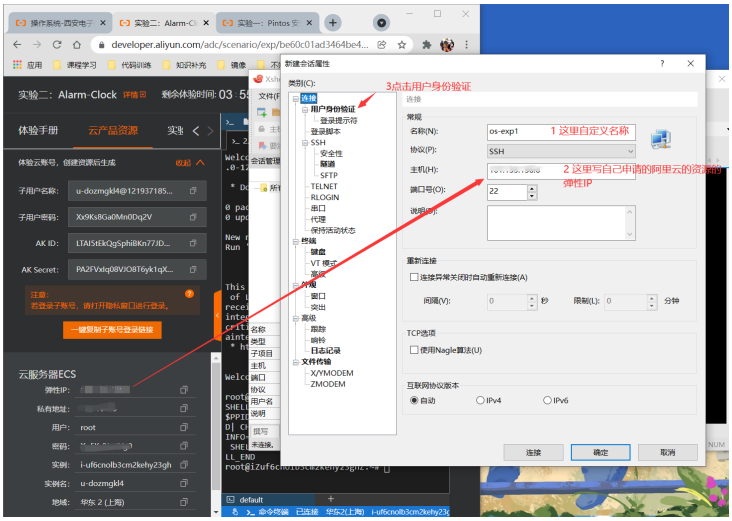


图 8: Xshell 软件界面-2

3. 然后点击用户身份验证，填入相应的身份表示信息

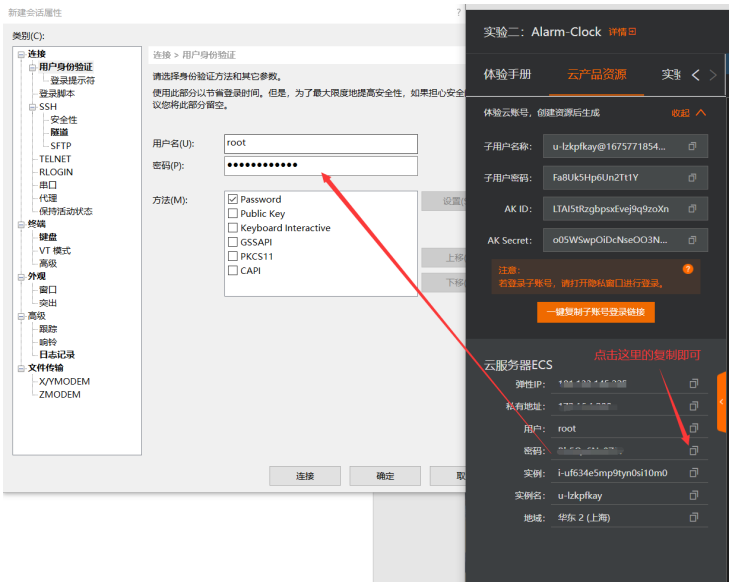


图 9: Xshell 软件界面-3

4. 出现这样的界面点击“接受并保存”，出现 *Welcome to aliyun Elastic Compute Sevice* 则表示远程连接成功了。

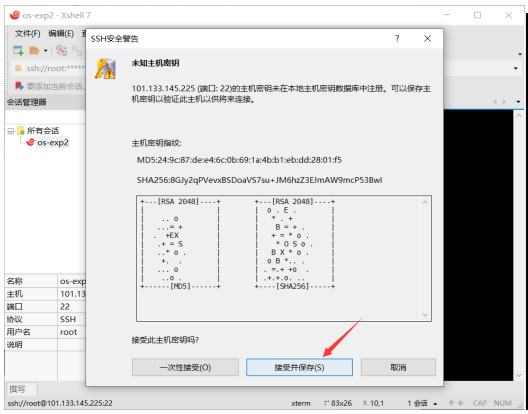


图 10: Xshell 软件界面-4

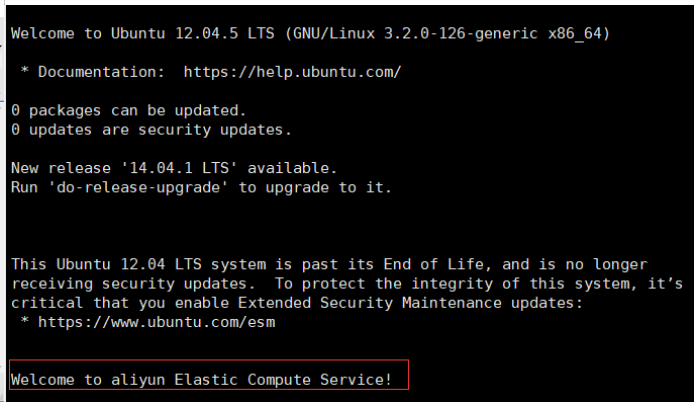


图 11: Xshell 软件界面-5

3.1.2　用 *VNC Viewer* 使用图形化界面登录

- 实验工具：*VNC Viewer*。
- 实验步骤：按如下步骤执行即可

1. 安装 *VNC Viewer* 后，打开软件，在相应的地址栏输入**弹性 IP 加上:1**

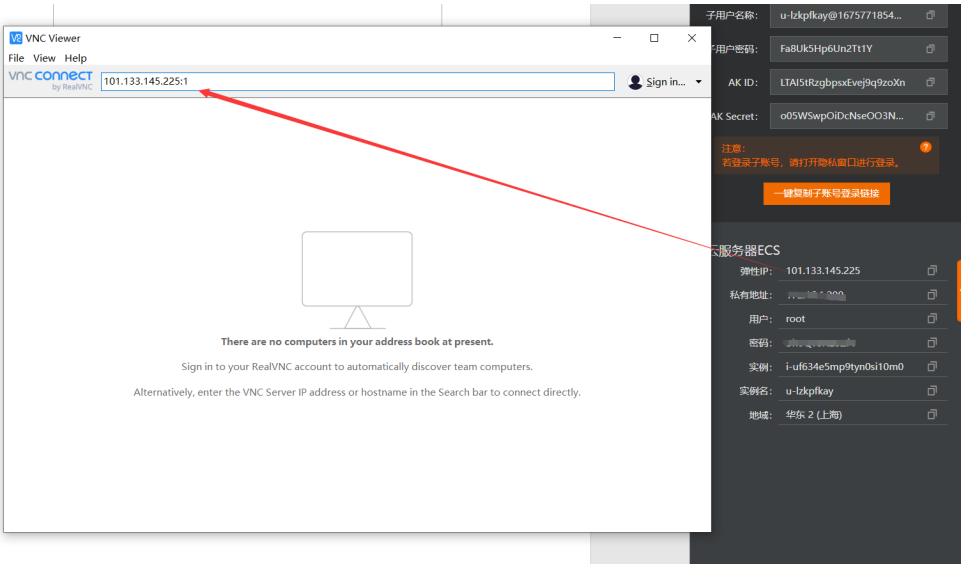


图 12: VNC 软件界面-1

2. 如果出现如下左图，直接点击 *continue* 即可显示右图，输入初始密码 *123456*

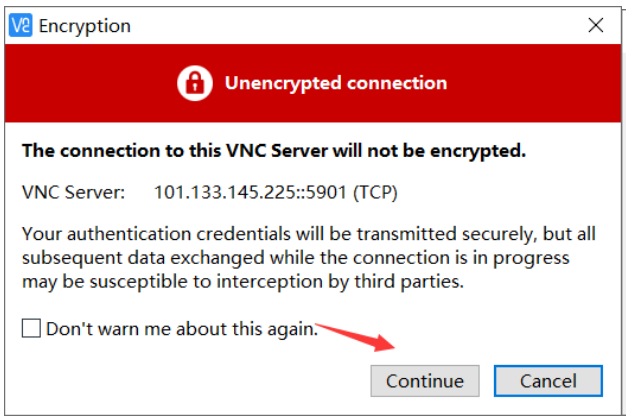


图 13: VNC 软件界面-2

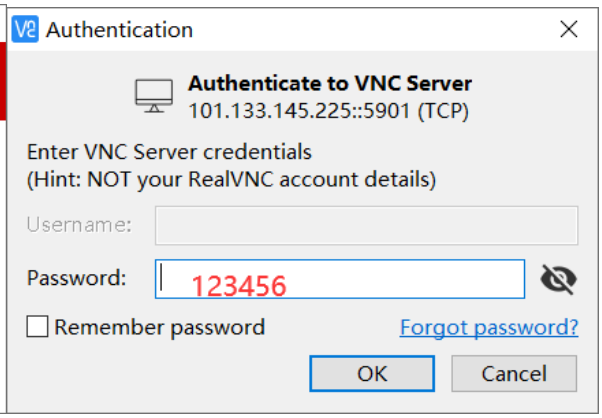


图 14: VNC 软件界面-3

3. 如果出现下图，一般这样的情况都是所连接的服务器没有配 *VNC* 的环境。

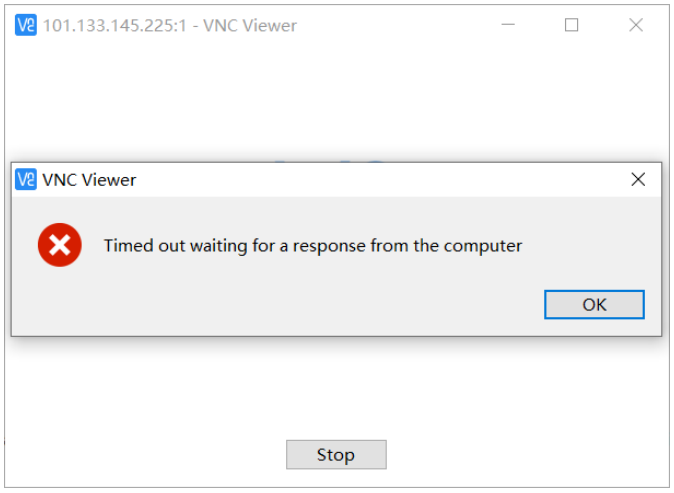


图 15: VNC 软件界面-4

所以需要先用 Xshell 按照 3.1.1 的云服务器的连接步骤连接服务器（Windows 用 Powershell 中的 SSH 也行，Mac 直接 Terminal 中 SSH 命令，本质都是一样的）进入命令行模式。在 *Xshell* 的命令行中依次输入:

- `vncserver :1`
- `vncserver -geometry 2560x1440`

注意:这里的‘x’是字母 x，不是‘*’，这里是调整分辨率，可以根据你的电脑显示屏的分辨率来调，可以低于电脑分辨率，不能高于分辨率，我这里的显示器是 2k 屏，所以是 2560x1440,1080P 的话应该就是 1920x1080 不然可能会出现颜色乱码。

注意:结束实验后，服务器会自动清除相关配置，所以在每一次体验创建资源后，都需要配置一遍 *VNC Viewer*，不然就会出现上图的 *Timed out*

4. 我们成功进入图形化界面, 我们的基本配置也算是结束了。

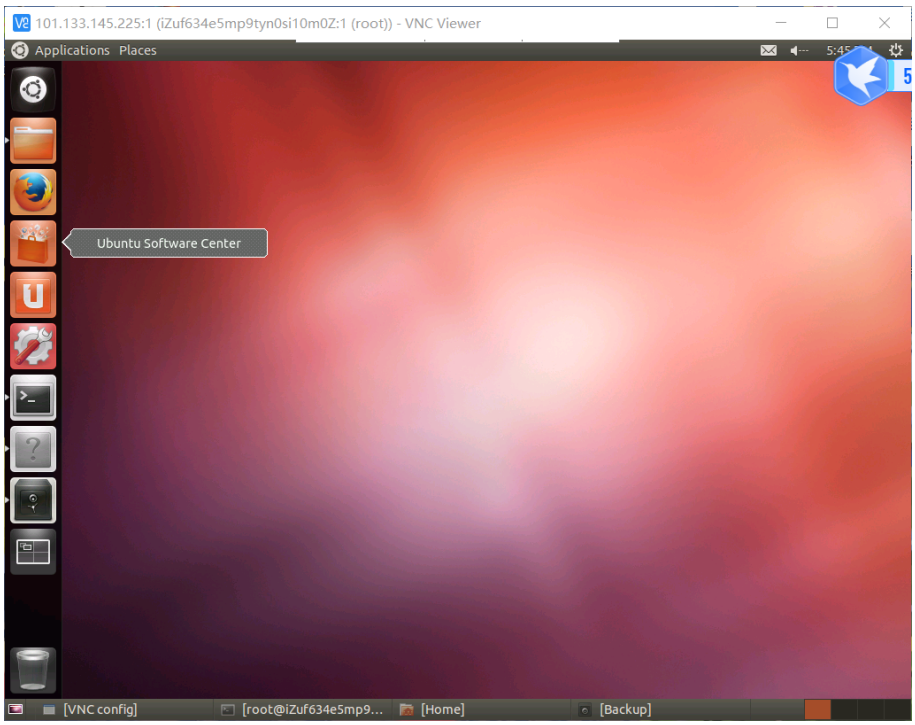


图 16: VNC 软件界面-5

注意：因为这是在云服务器上完成的，所以服务器已经给我们预装了 *bochs* 虚拟机以及安装在其中的 *pintos* 系统。可以在对应的*/home* 下文件夹下查看。如果没有下图的内容，则需要自己安装 *bochs* 和 *pintos*。

图 17: VNC 软件界面-6

3.2 正式实验

根据实验目的，基于对实验内容的充分理解的基础上，我们需要修改一下三个文件中的代码。

这里我们需要注意的是所有实验是一体的，所以在完成实验四的时候，如果是重新启动了云服务器，而不是在接着实验三后完成的，那么实验三中所需要修改的在实验四中也需要修改，这里只展示实验四需要修改的代码

文件名	路径
thread.h	*/pintos/src/threads*
synch.c	*/pintos/src/threads*
thread.c	*/pintos/src/threads*
synch.h	*/pintos/src/threads*
...	...

1. 在 *thread.h* 中加入结构体 *pri_stack* 用于保存被捐赠前线程的优先级以及被捐赠时捐赠者所需的锁。

```
struct pri_stack
{
    int priority;        //用于保存被捐赠之前的优先级
    struct lock *lock;   // 标明捐赠者在捐赠时需要的锁
}
```

2. 在 *thread.h* 中修改 *thread* 结构体，加入相应的变量。

```
struct pri_stack pri[64];
int top;                            //pri的栈顶
struct semaphore *block_sema;       //被哪个信号量阻塞
struct lock *block_lock;            //在等待哪个锁
```

3. 在 *thread.c* 中修改 *thread_set_priority()*

```
void thread_set_priority (int new_priority)
{
    /* 添加 */
    if(thread_current()->top > 0)
    {
        thread_current()->pri[0].priority = new_priority;
    }
    else
    {
```

```
        thread_current ()->priority = new_priority;
        /* 添加 */
        if(!list_empty(&ready_list))
        {
            if(new_priority <= list_entry(list_begin(&ready_list),
                struct thread,elem)->priority)
            {
                thread_yield();
            }
        }
        /* 结束 */
    }
}
```

4. 在 *thread.c* 中修改 *init_thread*

```
init_thread (struct thread *t, const char *name, int priority)
{
    ASSERT (t != NULL);
    ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
    ASSERT (name != NULL);
    memset (t, 0, sizeof *t);
    t->status = THREAD_BLOCKED;
    strlcpy (t->name, name, sizeof t->name);
    t->stack = (uint8_t *) t + PGSIZE;
    t->priority = priority;
    t->magic = THREAD_MAGIC;
    list_push_back (&all_list, &t->allelem);
    /* 添加 */
    t->top = 0;
    t->block_sema = NULL;
    t->block_lock = NULL;
    /* 结束 */
}
```

5. 在 *synch.c* 修改 *sema_down()* 函数

```
void sema_down (struct semaphore *sema)
{
    enum intr_level old_level;
    ASSERT (sema != NULL);
    ASSERT (!intr_context ());
    old_level = intr_disable ();
    while (sema->value == 0)
    {
        /* 添加 */
        thread_current()->block_sema = sema;//记录sema
```

```
        /* 删除 */
        // list_push_back (&sema->waiters, &thread_current ()->elem);
        list_insert_ordered(&sema->waiters,&thread_current ()->elem,thread_less,NULL);
        /* 结束 */
        thread_block ();
    }
}
```

6. 在 *synch.c* 修改 *sema_up()* 函数

```
void sema_up (struct semaphore *sema)
{
    enum intr_level old_level;
    /* 添加 + 修改 */
    struct thread *t;
    ASSERT (sema != NULL);
    old_level = intr_disable ();
    if (!list_empty (&sema->waiters))
    {
        t=list_entry (list_pop_front (&sema->waiters), struct thread, elem);
        thread_unblock (t);
        t->block_sema = NULL;//刷新满足了的线程的sema
    }
    sema->value++;
    intr_set_level (old_level);
    thread_priority_check();
    /* 结束 */
}
```

7. 在 *synch.c* 中修改函数 *lock_acquire()*

```
void lock_acquire (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (!lock_held_by_current_thread (lock));
    /* 添加 */
    if(lock->semaphore.value == 0)
    {
        thread_current()->block_lock = lock;
        priority_donate(thread_current(),lock);
    }
    /* 结束 */
    sema_down (&lock->semaphore);
```

```
    lock->holder = thread_current ();
}
```

8. 在 *synch.c* 中修改函数 *lock_release()*

```
void lock_release (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (lock_held_by_current_thread (lock));
    /* 添加 */
    if(thread_current()->top > 0)//被捐赠过
    {
        struct thread *t = thread_current();
        int i,j;
        while(t->top > 0 &&t->pri[t->top - 1].lock == lock)
        {
            t->priority = t->pri[t->top - 1].priority;
            t->top --;
        }
        for(i=t->top - 1;i>=0;i--)//找lock锁，作出相应操作
        {
            if(t->pri[i].lock == lock)
            {
                if(0<i&&i<(t->top-1))//不在数组底，直接删除
                {
                    for(j=i;j<(t->top-1);j++)
                    {
                        t->pri[j] = t->pri[j+1];
                    }
                    t->top--;
                }
                else if(i == 0)//数组底，赋值初始优先级后删除
                {
                    j=i;
                    t->pri[j+1].priority = t->pri[j].priority;
                for(;j<(t->top-1);j++)
                {
                    t->pri[j] = t->pri[j+1];
                }
                t->top--;
                }
            }
        }
    }
    /* 结束 */
}
```

9. 在 *thread.c* 中定义函数 *semaphore_elem*

```c
struct semaphore_elem
{
    struct list_elem elem; /* List element. */
    struct semaphore semaphore; /* This semaphore. */
    /* 添加 */
    int priority;
    /* 结束 */
};
```

10. 在 *thread.c* 和 *thread.h* 中对 *priority_donate(struct thread *t,struct lock *lock)* 函数进行定义：

Step 1: 将 *priority_donate(struct thread *t,struct lock *lock)* 函数加入到 *thread.h* 中

```c
void priority_donate(struct thread *t,struct lock *lock)
```

Step 2: 在 *thread.c* 中定义 *cond_less()* 函数

```c
void priority_donate(struct thread *t,struct lock *lock)
{
    struct thread *holder;
    if(t->priority > (lock->holder)->priority)
                        //优先级大于占用lock的优先级，则进行优先级捐献
    {
        holder = lock->holder;
        ASSERT(holder->top<64);//保证小于64
        holder->pri[holder->top].priority = holder->priority;
                //保存占用lock的优先级
        holder->pri[holder->top].lock = lock;
                        //保存捐赠时对应的lock
        holder->top ++;
                                                //栈顶自增
        holder->priority = t->priority;
                                //捐赠
        if(holder->status == THREAD_READY)
                                //若在readylist中，重排
        {
            list_remove(&holder->elem);//delete
            list_insert_ordered(&ready_list,&holder->elem,thread_less,NULL);
        }
        else if(holder->status == THREAD_BLOCKED &&NULL != holder->block_sema)
        //若占有lock的线程也在等待sema，在waiters重排
        {
```

```
            list_remove(&holder->elem);
            list_insert_ordered(&(holder->block_sema)->
                waiters,&holder-
            >elem,thread_less,NULL);
            if(NULL != holder->block_lock)
            {
                priority_donate(holder,holder->block_lock);
            }
        }
    }
}
```

3.3 实验结果

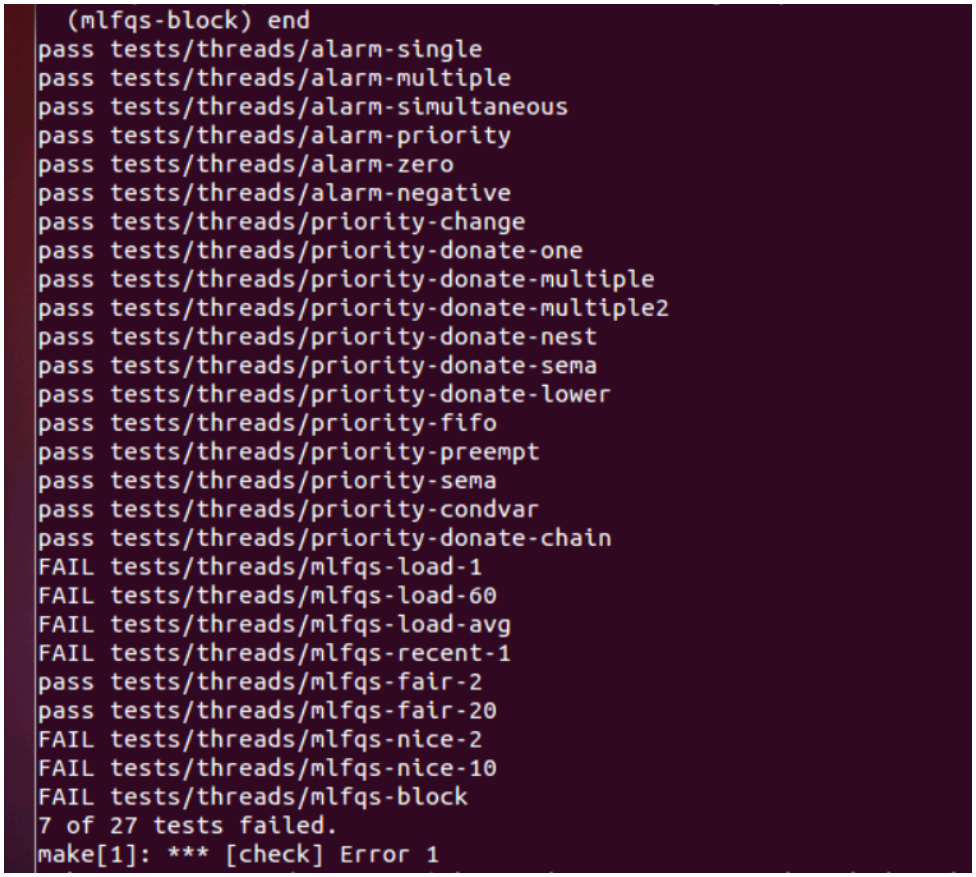
在 *teriminal* 中进入*../pintos/src/threads/*目录，输入：

```
make check
```

观察到只有以下 7 个检测不通过，其余均为通过则为实验成功！

- *mlfqs-load-1*
- *mlfqs-load-60*
- *mlfqs-load-avg*
- *mlfqs-recent-1*
- *mlfqs-nice-2*
- *mlfqs-nice-10*
- *mlfqs-block*

如下图所示



```
  (mlfqs-block) end
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-chain
FAIL tests/threads/mlfqs-load-1
FAIL tests/threads/mlfqs-load-60
FAIL tests/threads/mlfqs-load-avg
FAIL tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
FAIL tests/threads/mlfqs-nice-2
FAIL tests/threads/mlfqs-nice-10
FAIL tests/threads/mlfqs-block
7 of 27 tests failed.
make[1]: *** [check] Error 1
```

图 18: 结果截图

4 实验心得

通过本次实验，我粗略地了解了 *pintos* 的简单内核代码的含义，了解了有关线程优先级的定义和线程工作之间的调度关系，本次实验是通过线程的优先级捐献来解决因为 lock 造成的优先级反转的问题，从报告中可以看出优先级捐献总共有三种方式：简单捐献、多重捐献、递归捐献，每一种情况在程序的编写中都必须考虑到，代码的编写过程中我借助了老师提供的代码资料和课程讲解，仔细阅读和聆听之后，遇到很多问题，最终借助网络上他人的解答和经验解决了问题，完成了最终的实验，实现了应该出现的正确结果。从中学到了很多有关优先级的捐献的知识，同时要间接地学会了一些 c 语言代码编写的技巧。